2026.04.11

220243659 李星峰 每周汇报 （3.28-4.10）

一、实验情况

1.完成 `BackboneWrapper` 类的Fast NAS剪枝流程，在完整的univ2x_coop_e2e车路协同模型上的迁移。能够实现对模型的车端和路侧模型的 `img_backbone` 部分的同时剪枝，有以下问题和现象：

- [ ] **原始基准模型配置文件更改，baseline模型的backbone为关闭DCN层的resnet101**。因为对 `DCN层` 的剪枝方法仍未想出合适的剪枝规则或方法，原版 `backbone` 中 `resnet101` 的第三和第四 `stage` 中的 `DCN` 仍**保持关闭**，该部分网络结构和权重初始化保持与 `resnet101` 原始一致。
- [ ] **backbone剪枝微调后的模型具备恢复甚至反超原始模型（resnet101带有DCN层）的潜力，即使微调时使用的是最简单的原始模型配置文件的训练逻辑。**即便对 `resnet101` 中不带有 `DCN层` 的模型的 `backbone` 进行剪枝，经过3-6个轮次的微调后，模型的检测精度基本都能恢复到原始 `resnet101` 中带有 `DCN层` 的模型**几乎一致的水平**（相差最大不超过0.1），甚至有的**微调后的模型比原始模型的检测精度还稍高**。该现象在上次周报中对 `inf_track` 模型的backbone剪枝实验中已经指出，对 `veh_track` 和 `coop_e2e` 模型的 `backbone` 剪枝实验时同样发现该现象。
- [ ] **UniAD的tiny模型backbone权重替换进UniV2X的veh_track模型中不适用,检测效果衰退.** `UniAD` 的项目网站上没找到 `tiny` 模型的权重和配置文件，在 `Bench2Drive` 中找到了一个。用 `uniad_tiny` 模型的 `backbone` （网络结构为**resnet50**）权重替换univ2x的veh_track模型的backbone部分的权重，配置文件中的 `img_backbone` 部分也换为 `uniad_tiny` 配置文件的相应内容，发现**各类目标检测精度全部衰退为0**。对比两份配置文件 `bev_h` 与 `bev_w` 不一致，且bev编码器 `BEVFormerEncoder` 的层数 `num_layers` 也不一致，也许是 `bev_embed` **编码失真**导致检测器失效。

2.完成对 `veh_track` 模型的 `track_head` 的 `FFN` 中的 `linear层` 的剪枝实验,锁定输入输出接口层的维度,且不对注意力头进行剪枝,只对FFN层剪。类比对 `backbone` 包装的逻辑写了针对模型中 `track_head` 进行包装的 `Track Head Wrapper` 的类，将track_head解码器用于检测的分类 `track_head.cls_branches` 和回归 `track_head.reg_branches` 以及用于轨迹预测回归 `track_head.past_traj_reg_branches` 的linear层锁上 `locked` ，对多头注意力 `MultiheadAttention` 中网络层通道和头的个数也锁上了，只对FFN层的 `linear` 纳入剪枝搜索范围.

3.完成对 `veh_track` 原始模型（backbone去除DCN层）、剪枝 `backbone/track_head` 、微调3轮次模型的每帧推理时耗的评估，记录其中一些关键函数的耗时如下表所示（以剪枝 `track_head` 及微调为例）

耗时（ms）	original	pruned	finetuned
backbone	34.543213	35.764204	31.958456
neck	1.524004	1.622754	1.345787
track_head	78.348235	79.617847	69.590621
seg_head	92.522569	101.880657	75.635801
bev_encoder	63.423446	65.244882	54.29922
track_head_decoder	14.924789	14.372965	15.291401
extract_img_feat	36.183052	37.490155	33.421618
get_bevs	99.631339	102.756679	87.747013
velo_update	1.451339	1.223214	1.295992
track_base_update	140.793396	178.164025	87.565844
memory_bank	2.348044	2.500073	2.00073
query_interact	2.359107	2.556212	1.919706
single_frame_infer	271.3731	312.903363	203.625862
simple_test_track	273.921841	314.34112	204.862718
e2e	390.878963	438.94447	306.538379

track_head=bev_encoder+track_head_decoder

extract_img_feat=backbone+neck

get_bev=extract_img_feat+bev_encoder

但 **e2e=backbone+neck+track_head+seg_head+velo_update+track_base_update+memory_bank+query_interact+35ms**,每帧推理时的**四个模块和更新的时耗总和并不与e2e总耗时相等**,**大概总会差35ms的时间,**误差来源尚不清楚,可能是还有其他GPU计算的操作未能完全统计或者推理时**数据集加载**的时耗未能统计.

二、文献阅读和思考分析

仔细梳理了**BEVformer\spareBEV\PruneDrive**三篇论文.

虽然BEVformer没有着重提到稀疏化和加速计算,但其实它使用的**BEVformerTrackHead**中,无论是编码器部分的**TSA**和**SCA**,还是解码器中用于输出检测框的注意力,都其实是基于稀疏采样点(8个 `head` ,**TSA**中为1个 `level` ;**SCA**中为4个 `level` 中,每个头每 `level` 4个2D采样点 `2d_ref` ,**SCA**中由200*200个 `BEVquery` 每个在z轴均匀取4个3D参考点 `3d_ref` 在不同相机视图上投影产生)交互的**多头可变注意力机制** `Deformable Attention` ,并不是典型的 `Transformer` 的全局注意力,编码产生 `bev_embed` 时,每个 `BEVquery` 只与稀疏**采样点**(由 `BEVquery` 的 `feature` 通过 `linear` 层预测的

`offset` 偏移量产生)对应的 `value` 做计算,且**注意力权重**也不是通过Q与K转置点乘产生,而是同样由 `BEVquery` 的 `feature` 通过 `linear` 层预测得到.

1.网络结构和query构造方面:spareBEV对我的感觉,像是摒弃了**BEVformer**中 `track_head` 中的**编码器**部分,不再使用稠密的200*200的**BEV网格的编码** `bev_embed` 表示,直接只使用900个query直接对**BEV场景**进行建模,像是**只保留了解码器部分.**但对 `query` 加上了**几何先验的检测框(中心点\长宽高\速度\朝向)**等信息,形状也直接拉成了**柱状,**再加挂一个D维的 `query feature` **.**

2.注意力机制方面:其中像**编码器**的部分,是将**可变注意力**改为了**SASA**(带与距离相关的可学习参数**惩罚因子**),又和可变注意力机制中预测的**偏移量** `offset` 很像,区别在于**spareBEV**只是对自注意力机制做了距离惩罚偏置,改变了**注意力的权重**,但不改变 `query` 间两两交互做自注意力的计算数量,本质上还是做**全局注意力计算**;**BEVfomer**中可变注意力是每个 `query` 只对预测出的少量采样点对应的 `query` 进行权重聚合计算,**注意力权重**也是对 `query` 通过 `linear` 直接预测出来的.

3.时空特征融合方面:spareBEV像是把**BEV former**中**编码器**中的**TSA**和**SCA**一次性全搬进了**解码器**中,只不过**BEV forme的TSA**在迭代更新 `BEVquery` 为其注入历史信息的时候,维护了 `prev_bev` 和 `bev_state` 两个队列,推理时都将前一帧的 `BEVquery` 压缩进 `prev_bev` ,这种记忆交互方式很像**RNN**的更新状态，不断将所有的历史信息压缩进同一潜空间内的中间表示.在**SCA**时再用 `BEVquery` 在 `z轴` 上均匀取的4个点为3D参考点向**当前帧的不同相机视图**投影,再对不同空间视图下的信息进行融合.相当于**BEVformer一共生成了两次参考点,第一次时TSA中对BEV图的2D参考点,第二次是SCA的3D参考点进而向不同视角的相机图投影得到２Ｄ参考点,究其根本两次的注意力机制都是可变注意力,本来就算依靠所生成的稀疏参考点+偏移量offset来计算的**；而**spareBEV**在**Adaptive Spatio-temporal Sampling**中融合历史信息的方式则更加简单,直接**将生成的3D采样点**投影到当前帧和此前 `8` 帧的所有相机图上,query直接通过所生成的3D采样点的投影点与历史 `8` 帧的所有相机图像进行交互,相当于**一次性**利用**BEVformer**中的**SCA**完成对**时间历史信息**和**空间多视角信息**的融合.参考点 `ref` 做相机图的投影之前,两者都按 `ego pose` 做了**自车运动对齐,BEVformer**中是在**TSA**的有 `prev_bev` 路径下用 `shift_2d_ref` 完成的,**spareBEV**对 `query` 的速度还做了**目标运动对齐**,都是在自适应采样时完成的.

从时序信息利用上看,BEVformer似乎只利用了当前帧 `bev_state` **和**前一帧 `prev_bev` **信息,即便**前一帧**的** `prev_bev` 其实压缩进了所有历史帧的信息(但可能也会像**RNN**面临长时序记忆**稀释**问题),而spareBEV的query是直接对 `8` 帧的原始历史相机图进行交互,而不是对被不断压缩进同一潜空间内的中间变量进行交互.但实际上**BEVformer**中也有 `queue_length` 的变量,可以控制**编码器**一次输入的帧数,在训练时同样可以直接在编码器中一次利用**多帧历史图像**输出具有更好质量的 `prev_bev` ,从而得到更高建模质量的 `bev_embed`

Prune Drive是对用于自动驾驶的VLM模型的视觉Token剪枝,主要有以下两个关键模块:

1. T-FPS: 多样性感知的 token 选择

借鉴的是点云里的 Farthest Point Sampling (FPS)，把它改造成视觉 token 选择方法 **T-FPS（Token-wise FPS）**。传统 FPS 的思想是,每次不是选“最显眼”的点，而是选“离已经选中的集合最远”的点，

这样最终保留下来的点能更均匀、更有覆盖性。论文把这个思想搬到视觉 token 上，不再用欧氏距离，而是用 `token embedding` 空间中的 **cosine distance**。先随机选一个 token,对所有**未选 token**，计算它到**已选集合**的**最小**距离,再选出那个**离已选集合最远**的 token。最后得到的不是一堆高注意力但可能彼此重复的 token，而是一组**语义上更分散、空间上更有覆盖**的 token。不是“保留**最重要**的几个 token”，而是“保留**最有代表性**的一组 token”。

2. View-adaptive pruning：按视角自适应分配保留比例

第二个核心部件是视角级 token 预算分配。作者认为，**6 个摄像头对下游驾驶任务的重要性并不相同，所以不应该给所有视角分配同样的 token 保留率。**因此他们把“**每个视角保留多少 token”写成一个优化问题。**设每个视角的保留率为 α={α1,…,αM}，目标是在给定预算下最大化一个目标函数 M(α)。这个目标函数是一个奖励-惩罚结构：

奖励项 R(α)：衡量剪枝后模型输出和真值之间的任务表现相似度；

惩罚项 P(α)：所有视角保留率之和，鼓励更稀疏、更省算力；

λ：控制性能和效率的权衡。

哪些视角该多留，哪些视角可以多剪，由验证集上的优化自动搜出来。 搜索时用的是冻结的预训练 VLM，在一个从训练集随机抽取的小代表性子集上完成，以降低代价并避免泄露。
对于相同总 token 预算，“视角自适应预算分配 + T-FPS 多样性采样” 会比 “统一保留率 + 随机采样” 取得更紧的误差上界。

三、思考与启发

spareBEV的**计算稀疏化**,本质上在于它直接**切除了复杂的BEVencoder**,**放弃对稠密BEV图的编码构建**,直接用解码器中**稀疏的900个query**对BEV场景进行建模和输出检测预测结果。

但相应的代价是它在**SASA**中对初始query进行交互更新时又恢复了稠密的自注意力计算,即便它说加入了距离相关的惩罚因子,但也只是对注意力权重大小的影响,并不改变实际参与计算的query数量。

此外,在**生成3D参考点进行采样计算方面**，BEVformer的track_head在编码器和解码器中相当于调用可变注意力机制进行了**三次参考点生成**（编码器中TSA和SCA两次，解码器一次），而spareBEV只是在解码器的自适应采样中进行一次。更具体来说，BEVformer编码器中，每个BEVquery生成的参考点对每个level都会生成4个偏移采样点，**TSA**是2D的BEV图level为１，生成的是**40000**个2d参考点，每个query最多可能与**8head*1level*4point=32**个采样点对应的value计算。**SCA**是空间相机图的特征图level为4，生成的是**40000*4=160000**个3D参考点，也就是说**假设参考点全部命中6个视图**，会有**6view*4ref（每个BEVquery在z轴均匀取点数）*8head*4level*4point=3072**个采样点对应的value与每个query计算。

spareBEV在**生成3D参考点时便进行偏移量预测，每个query固定每帧16个点，对8帧6视角图像投影，**全部命中，会有**6view*8head*8帧*16point=6144**个采样点对应的value与每个query计算**.简单来看，从采样点个数来看，spareBEV的采样点其实是大于BEVformer的，**但**BEV former**的编码器中是密集的40000个BEVquery进行计算，**TSA+SCA= 40000*（23+3072）=124,160,000，spareBEV为**

SASA+自适应采样=900*（900+6144）= 6,339,600，是前者的大概5%左右，前者BEV former还没算上解码器部分的可变注意力计算。

结合**PruneDrive中对视觉编码token的裁剪，我觉得很值得对UniV2X模型借鉴的部分，有三个部分：**

1.BEVformer编码器的BEVquery数，会直接改变bev_embed维度,但本身该模型就依赖于稠密的BEV场景表示，直接修改这种后续各任务头都需要用到的统一场景表示的bev_embed风险很大，暂未考虑这样做，但这是跟PruneDrive对token过滤的做法最为相似的地方。

2.BEVformer编码器中TSA使用的可变注意力机制的改造。借鉴 SparseBEV 的 SASA 思路，历史时序语义关联性较强的是本身距离较近的目标。对可变注意力**添加距离相关的惩罚项**，直接收缩offset，局部性越强，偏移半径越小；惩罚远点权重，允许模型去远处看，但远点默认更难拿到大权重；再对历史bev分支**history branch**添加额外正则项，根据运动状态和几何信息修正注意力窗口范围大小。

$$\hat{\Delta p}_{i,h,k} = e^{-\gamma \tau_{i,h}} \Delta p_{i,h,k}$$

$$\hat{a}_{i,h,k} = a_{i,h,k} - \lambda_{\text{dist}} \tau_{i,h} \|\hat{\Delta p}_{i,h,k}\|_2^2$$

其中：

- γ 控制对 offset 的直接收缩强度
- λ_{dist} 控制对远点权重的抑制强度

$$L_{\text{loc}} = \lambda_{\text{loc}} \cdot \frac{1}{BHNK} \sum_{i,h,k} w_i \|\hat{\Delta p}_{i,h,k}\|_2^2$$

其中 w_i 可以按运动状态自适应：

$$w_i = \exp(-\beta \|v_i\| - \eta |\dot{\psi}_i|)$$

意思是：

- 低速、小转角：更强调局部性
- 高速、大转角：放松局部性

3.BEVformer编码器中SCA阶段的生成的3D参考点数量，把 Prune2Drive 的视图裁剪 + 两阶段偏移引入SCA.原则是不同视图不该接收同样多的2D初始投影参考点,且每个视图上预测出的采样点应覆盖更大范围,取得的特征应更具代表性.

分为两层,**第一层视图自适应门控**,保持总生成3D参考的数量一定的情况下,控制6个视角相机图中可接收到的2D初始投影点数,通过各视角下的重要性分数,保留不同视角下保留的命中的2D投影参考点数量或比率,像PruneDrive那样写成一个优化约束问题.

对第 i 个 query、第 c 个 camera，先计算一个 view score：

$$s_{i,c} = f_{\text{view}}(q_i,\ e_c,\ \rho_{i,c},\ \bar{z}_{i,c},\ \bar{u}_{i,c},\ \bar{v}_{i,c},\ m_i)$$

其中：

- q_i：BEV query feature
- e_c：camera embedding
- $\rho_{i,c}$：该 query 的 3D anchors 在 camera c 上的命中比例
- $\bar{z}_{i,c}$：平均投影深度
- $(\bar{u}_{i,c}, \bar{v}_{i,c})$：平均 2D 投影位置
- m_i：运动状态

第二层:两阶段偏移，但保持总点数不变.原始 SCA 每个 level 采 4 个点,虽然足够稀疏,但因为在每个level都分了4个采样点,其实可以更为细致地分为远近结合的两阶段粗细采样,第一阶段用两个粗放点尽可覆盖更大的视野范围,第二阶段再用两个精细点精确刻画临近关联点间的语义信息,总点数和原始 SCA 一样,不会显著增加计算开销,也可以在不同的level保留不同的粗放\精细点数

2.4 第一阶段：粗偏移

对保留下来的 camera c 和 level l，预测 coarse offsets：

$$\Delta p_{i,c,l,j}^{(1)} = \sigma_{\text{coarse}} \cdot \tanh\left(W_l^{(1)}[q_i; e_c]\right), \quad j = 1, 2$$

得到 coarse sampling points：

$$p_{i,c,l,j}^{(1)} = r_{i,c,l} + \Delta p_{i,c,l,j}^{(1)}$$

其中 $r_{i,c,l}$ 是该 query 投影到 camera c、level l 的初始 2D 参考点。

为了让 coarse points 更分散，我建议加一个覆盖损失：

$$L_{\text{spread}} = \lambda_{\text{spread}} \sum_{j \neq j'} \exp\left(-\frac{\|p_j^{(1)} - p_{j'}^{(1)}\|_2^2}{\sigma^2}\right)$$

这个损失小的时候，说明 coarse points 更分散。

2.5 第二阶段：细偏移

围绕每个 coarse point 再预测细偏移：

$$\Delta p_{i,c,l,j,t}^{(2)} = \sigma_{\text{fine}} \cdot \tanh\left(W_l^{(2)}[q_i; e_c; f_{i,c,l,j}^{(1)}]\right), \quad t = 1, 2$$

其中 $f_{i,c,l,j}^{(1)}$ 是第一阶段 coarse point 采到的局部 feature。
然后：

$$p_{i,c,l,j,t}^{(2)} = p_{i,c,l,j}^{(1)} + \Delta p_{i,c,l,j,t}^{(2)}$$

这里约束：

$$\sigma_{\text{fine}} < \sigma_{\text{coarse}}$$

$$\downarrow$$

让第二阶段天然更局部。

2.6 最终聚合

对所有保留下来的 camera、所有 level、所有 fine points，计算权重：

$$a_{i,c,l,j,t} = W_a[q_i; e_c; f^{(1)}_{i,c,l,j}]$$

然后：

$$\alpha_{i,c,l,j,t} = \mathrm{softmax}_{c,l,j,t}(a_{i,c,l,j,t})$$

最终输出：

$$y_i = \sum_c g_{i,c} \sum_{l,j,t} \alpha_{i,c,l,j,t} \, V_c^{(l)}(p^{(2)}_{i,c,l,j,t})$$

这里 $V_c^{(l)}(\cdot)$ 是第 c 个 camera、第 l 个 level 的 bilinear sampled value。

四、下一阶段目标

1.实现基于对采样点的裁剪过滤门控机制

2.BEVquery的稀疏化过滤